CONCERTO API INTEGRATION GUIDE

DOCUMENT HISTORY

Version	Date	Author	Comments
1.0	05th Nov 2024	Dan Ross	Initial Draft
1.1	20th Jan 2025	Dan Ross	Additional APIs, PAT token notes and relationships
2.0	17th Apr 2026	Dan Ross	Hub OAuth 2.0 details, proxy details, recipes
2.1	29th Apr 2026	Dan Ross	Added detail on Input Types
2.2	6th May 2026	Dan Ross	ETA Action Inputs added
2.3	16th Jun 2026	Dan Ross	Added quote id input, removed proxy docs
2.4	22nd Oct 2026	Dan Ross	Added sample notes

TABLE OF CONTENTS

Document History 1
Table of contents 1
Introduction 2
  Key Terms 2
Functional Coverage 2
Overview 3
  APIs are generally read-only 3
  Use Push Events, Not Polling 3
  Different Integration Approaches 3
  Selecting an Approach 3
  Authentication 4
Push Events 5
  Signature 5
  Authentication schemes 6
  Event Type 6
  Additional Event Types 6
  Deduplicating Events 6
Key API Objects and Relationships 7
Actions API 8
  Running an Action 8
  Example Flow: Execute an Action 8
  Inputs and Payloads 9
  Input Types 9
Key Relationships 9
OpenAPI Specs 11
RecomMended Approaches 11
  Receive new work orders from Concerto 11
  Run Actions in Concerto 13
Appendix A: Input Types 15

INTRODUCTION

The Concerto APIs provide you with all the tools needed to manage orders throughout their operational lifecycle. The 'Functional Coverage' section below describes the functionality available via these new APIs. Any functionality not mentioned in the 'Functional Coverage' section below is out of scope.

Out of scope functionality not provided by these APIs at present includes, for example, Job Costing and Invoicing.

Depending on demand, functional areas such as these may be included in future iterations of the API. If you have use cases which require additional API models, fields or endpoints beyond what is described here, please let us know.

KEY TERMS

You'll find a few domain specific terms in this document – here are the key ones:

Term	Meaning
Order (or Works Order, Project Order)	A request from Bellrock that a supplier performs a task (a reactive request, a PPM, an accepted quote)
Helpdesk Job	A problem statement from a client letting Bellrock know that an issue needs resolving. A Helpdesk Job is solved when an Order associated with it is completed
PPM	'Planned' works – such as a fixed wire inspection
Statutory PPM	A PPM configured such that a certificate document is required before an associated order can be completed
Equipment Item	A physical item, often with a barcode attached
Equipment Type/Sub-type	A 2-level logical hierarchy for categorising Equipment Items
Helpdesk Classification	An n-level deep hierarchy used to classify Helpdesk Jobs
Site	A physical location with an address belonging to a Client
Block	A building or named area on a site (eg. 'Main Building')
FloorLevel	A floor within a Block (eg. 'Basement')
Area	A named area within a Floor (eg. 'Classroom 7')
Portfolio	A top-level logical grouping of customers. Often used to represent the 'group' level of multinationals or parent companies (eg. Super Everything Group Plc). Often a one-to-one relationship with Client.
Client	A legal entity-level customer. Often used to represent a 'brand' (eg. Super Manufacturing Ltd)
Tenant Key	A value used to route API requests around. Usually derived from the Portfolio.
Concerto Instance	A deployed copy of the Concerto platform, often housing a single Portfolio, or group of related Portfolios.

FUNCTIONAL COVERAGE

The APIs described in this document seek to cover the following broad functional areas:

Area	Functionality	Equivalent front end area
Orders	Find orders	Order Register, Supplier Portal
	View order details	Order Register, Supplier Portal
	Execute Supplier Order Actions	Supplier Portal
Quotes	Find quotes	Quotes, Supplier Portal
	Update quote details	Quotes, Supplier Portal
	Submit Quotes	Quotes, Supplier Portal
	Execute Quote Actions	Quotes, Supplier Portal
PPM	View PPM Types	PPM, Supplier Portal
	View Site PPM Records	PPM
Locations	Get details on Site, Block, Floor and Area	Sites
Equipment	Get details on Equipment Item	Equipment
	Get details on Equipment Types	Equipment
Documents	Retrieve document content	Various
Helpdesk	Find Helpdesk Jobs	Helpdesk
	View Helpdesk Job detail	Helpdesk
Actions	Expose configurable actions	Various

OVERVIEW

The APIs outlined in this guide are exposed by a given Concerto Instance. Each Instance is a self-contained independent universe which defines 'global' objects such as Helpdesk and Order Statuses, Actions, Portfolios and other 'reference data' used in other APIs.

Where an integrator needs to interact with multiple instances (perhaps because they do work for multiple clients), it's important that API requests are made against the correct Concerto instance. There are several approaches you can employ to make sure you're hitting the correct instance, outlined later.

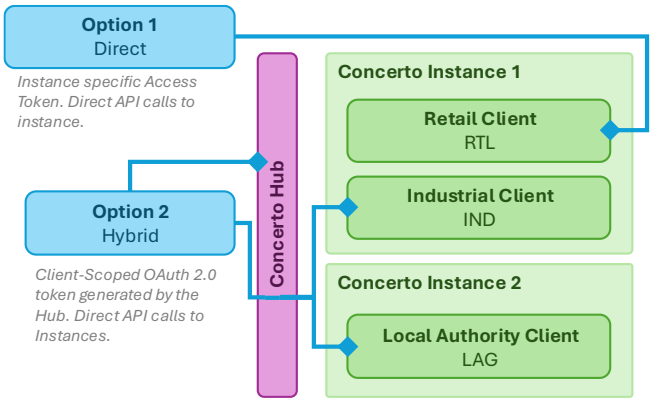
APIS ARE GENERALLY READ-ONLY

The APIs described here are generally read-only, and are there to provide details on data stored within Concerto. The notable exception to this is the Actions API (described below), which is used to execute Actions defined in Concerto against things like Orders or Quotes. The Actions API is the preeminent way to perform operations which mutate the state of data within Concerto.

USE PUSH EVENTS, NOT POLLING

Our APIs are designed to use our Push events to deliver updates to your platform in a timely and reliable manner. We strongly discourage polling for changes on a timed basis – this introduces considerable performance overhead and time lag in your integrations.

We employ quota based performance management behind the scenes, with your quota calculated and refreshed based on system load. You are very unlikely to ever hit your quota if using our APIs as described in this document. Aggressive polling for changes will likely trigger our quota management routines and you'll receive a `HTTP 429` until your quota regenerates.



DIFFERENT INTEGRATION APPROACHES

Option 1: Direct Instance Integration

If you only need to integrate with data for a single client, you can skip the complexity of the Hub and make direct calls to a specific instance, using a simple long-lived access token local to that instance. This simplifies your integration considerably and removes the need to partition reference data by client on your side.

🌟 **Best for:** when you integrate with a single client only

- ✅ Single API URL for your requests
- ✅ Simplest integration option
- 🤔 Non-standard authentication (long lived opaque bearer token)
- 🤔 Additional effort to integrate with other client instances in the future

Option 2: Hybrid Integration

Our recommended approach. Use the Routing API from the Hub to discover which instance a given client exists on, then use the OAuth 2.0 client credentials flow to obtain an instance scoped token from the Hub you can then use to make authenticated calls directly on an instance.

🌟 **Best for:** when you integrate with more than one client (or think you might need to)

- ✅ Standard OAuth 2.0 token flow and refresh
- ✅ Clean separation between instances
- ✅ Rely on our routing architecture, or maintain your own
- 🤔 Requires slightly more effort than direct instance integration

SELECTING AN APPROACH

Unless there is a compelling reason not to, **we suggest you adopt the Hybrid approach** outlined above and rely on the Routing API provided by the Hub to direct your API calls to the correct instance.

This approach future-proofs your integration – as new clients come on board you'll have already developed all the necessary infrastructure to support additional instances as and when they are deployed, without having to make sweeping changes.

If the Hybrid approach feels too much, consider instead creating a direct instance integration to cover your needs as they stand *right now* – it's quite practical to pivot to the hybrid approach later down the line.

AUTHENTICATION

Whichever integration approach you decide upon, you'll need to authenticate your requests.

DIRECT INSTANCE INTEGRATION

When making a request directly to a single Concerto instance, you will authenticate your requests by providing an Access Token (a PAT). A PAT is a long-lived opaque token you send up with each request, which grants you the identity of a specific user, and a subset of their permissions.

Provide your PAT as a `Bearer` token in a standard HTTP `Authorization` header.

If you've opted for this integration approach, we will provide you with a PAT you can use as part of an API handover session.

HYBRID INTEGRATION

A better approach to making requests to instances is to use an access token granted by the Hub.

In the Hub, create an API key – this will generate you a client secret. Use this with the 'client credentials' OAuth 2.0 flow with the following details:

Access Token URL:	https://hub-api.bellrock.co.uk/api/tokens/v1/token
Refresh Token URL:	Same as access token URL
Client ID:	Any value you like
Client Secret:	Your Hub API Key secret
Scope:	The base URL of the concerto instance you'd like to make calls to

REFRESHING YOUR ACCESS TOKEN

Post your existing token back to the original endpoint with a grant type of `refresh_token`, and a new token will be generated for you.

OBTAINING SCOPE VALUES

When using the hybrid approach, you'll need to know which instance of Concerto to scope your access token to. You can do this by providing the base URL of the concerto instance you plan on using as the Scope when generating your access token. Only one scope value is supported, so you'll need a different access token for each instance you'll be making requests to.

OBTAINING THE BASE URL FOR A GIVEN TENANT

You can discover which instance of Concerto to use by providing a Tenant Key to the Hub Routing API. This will return the base URL of the instance a given tenant is located on, which you can then use to generate a scoped access token.

Routing API URL:	https://hub-api.bellrock.co.uk/api/routing/v1/tenant/:tenant/baseurl
:tenant	The Tenant Key for the client you're interested in
Returns:	HTTP 200: A JSON object containing the base URL of the instance within which that tenant resides
	HTTP 404: The Tenant Key you provided is invalid

WHAT IS A TENANT KEY?

A Tenant Key is a simple string we use to route API requests around behind the scenes. You can obtain a tenant key by looking at the `portfolioCode` property of any order event you receive.

Why the multiple names? `portfolioCode` is just one of the different tenant keys we use across our APIs.

PUSH EVENTS

Polling of APIs described here is discouraged. Attempts to poll for data will likely result in your requests being rate limited. Changes to objects in Concerto result in a rich stream of events being generated which you can subscribe to.

Events are delivered as outgoing signed webhooks to an endpoint of your choosing, or a Service Bus topic hosted within your infrastructure.

The Web Hook you configure on this page will be invoked for every event broadcast to the channels you select as a HTTP POST request.

Triggered webhooks are queued and processed sequentially, so the delay between an event being generated and you receiving the HTTP request cannot be predicted. Do not rely on this web hook functionality for time sensitive notifications under any circumstances.

The event will be provided as a JSON payload in the body of the HTTP request with the following properties:

- id (guid) A unique ID for the event
- eventDateUtc (dateTimeString) The UTC Date and Time the event was generated
- channel (string) The channel the event was broadcast to
- type (eventType) The type of event
- data (object) Data related to the event. For order related events, the object will have the following properties:
- orderId (guid) The ID of the Order. You can use this with the /api/order/{id} endpoint of the HTTP API
- orderRef (string) The unique Ref of the Order. This is a human-readable ref you can use in the Supplier Portal

Any other properties can be ignored

SIGNATURE

Each request sent to your webhook carries a request signature. You can validate this signature to ensure the request you receive originated from Concerto and not a third party.

The signature is provided in the X-CAPI-SIGNATURE HTTP header and is based on a signing secret of your choosing, which is stored against your webhook configuration.

To verify the signature, do the following:

1. Create a Byte Array of size request.length + secret.length
2. Populate the byte array with the contents of request.body
3. Fill the remainder of the array with the bytes from your secret
4. Generate an SHA256 hash of this new array
5. Base64 encode the hash
6. Compare the value you created with the value of the X-CAPI-SIGNATURE header

If the values do not match, the request did not originate from Concerto and should be discarded.

If no Secret is specified, the header will be the SHA256 hash of the body

AUTHENTICATION SCHEMES

Authentication schemes such as basic auth or similar are not supported. Custom HTTP headers can, however, be appended to all outgoing requests. Discuss any requirements along these lines with your technical contact.

EVENT TYPE

The 'type' property of an event payload can have one of the following values:

- orderCreated : An Order has been Created
- orderAssigned : An Order has been Assigned to a Supplier
- orderAccepted : An Order has been Accepted by a Supplier
- orderRejected : An Order has been Rejected by a Supplier
- orderAttended : An Order has been attended by a Supplier
- orderCompleted : An Order has been marked as Completed by a Supplier
- orderCancelled : An Order has been marked as Cancelled by Concerto
- orderValueChanged : The Value of an Order has changed
- orderUpdated : A monitored field on the order has changed
- testEvent : A Test Event

ADDITIONAL EVENT TYPES

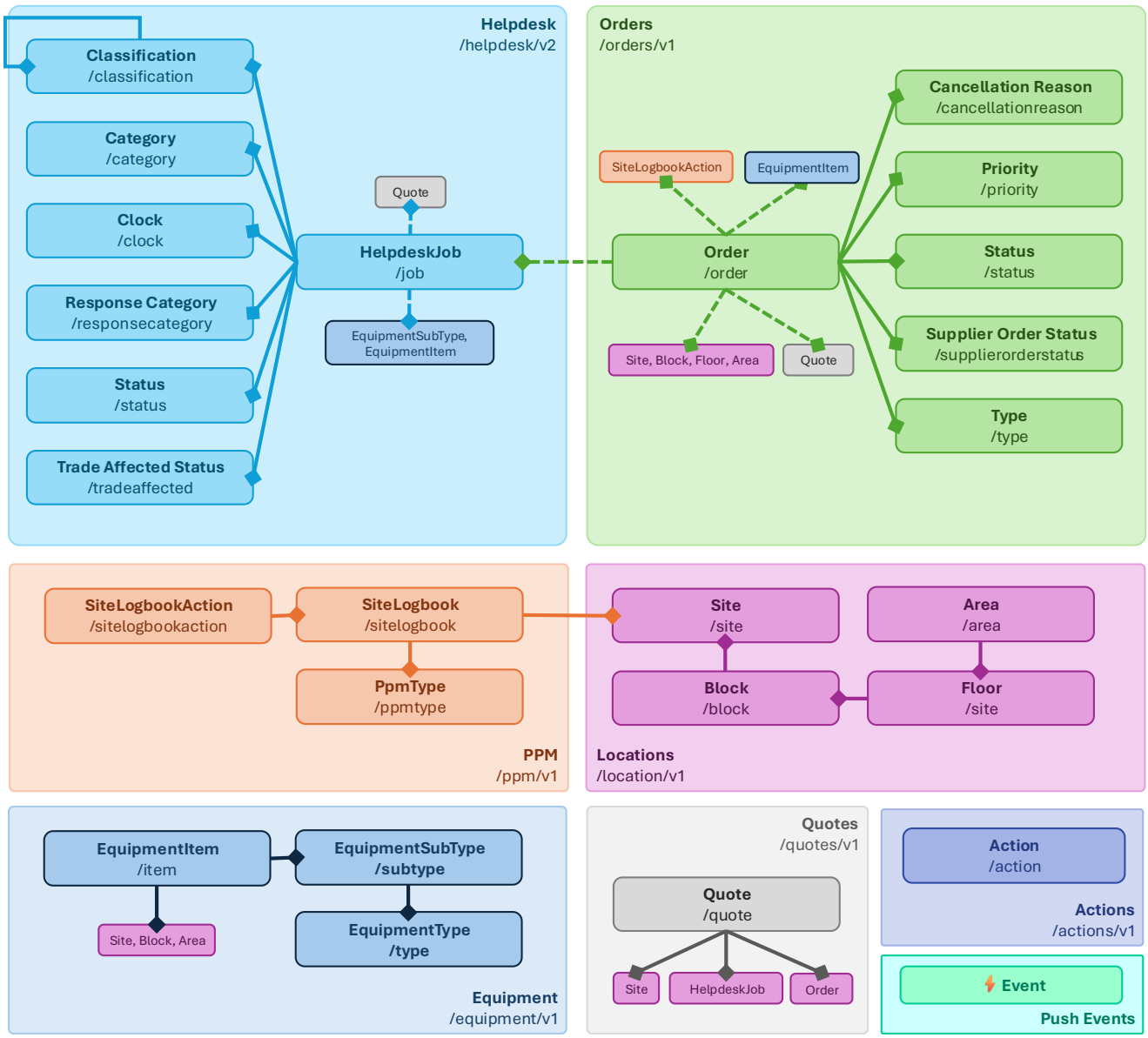
The above list is not exhaustive – new event types are being added all the time. Your implementation should support receiving event types other than those you are interested in

DEDUPLICATING EVENTS

Events are multicast to channels, and your web hook subscription may pick up the same event via different channels depending on the nature of the event, system configuration and your subscriptions. You should account for this in your design. You can de-duplicate events by looking at the ‘source’ and ‘event type’ properties of the event payload.

KEY API OBJECTS AND RELATIONSHIPS

The described APIs work together to provide a rich data model:



The API is primarily designed around core Order data - as this is the main transactional unit of interaction between an instance of Concerto and a Supplier. Other APIs generally provide additional details of objects referenced by an order – for example details on a Site on which an Order has been logged.

Several 'standard' endpoints exist where logical:

- /endpoint/:id/log endpoints will provide notes on a specified item (eg. /order/123/log for Order Notes)
- /endpoint/:id/document endpoints will provide attached documents (eg. /order/123/documents for Order Docs)

ACTIONS API

The Actions API is the means by which changes can be made to Orders and other objects in Concerto. Any requirement which involves 'running an Action' requires the use of the Actions API.

An Action is a user-configurable workflow step in Concerto which has a scope controlling the situations in which it can be invoked. The scope takes into account elements such as the type of Order, the current status of the Order, the user making the request and several other factors.

Many actions require data to be provided to an action for it to complete its work – for example the 'Add Note' action will require some content to record as the note.

The Actions API provides you with a means to:

- Discover actions available for an Order (or other object) given its current state
- Identify the Action you want to run by searching for a well-known key, name or ID
- See what data you will need to provide to run a given action against a given object
- Validate that a payload you want to pass to an action is valid
- Run an action

Actions declare the type of data they require in order to successfully execute by returning a collection of Inputs. An Input has a well-defined 'type', a flag to say whether this input is mandatory, and optionally some metadata to help better define what a valid input might look like.

This API can be quite complex to consume in a robust manner – so we've created a model implementation for you here: https://github.com/BellrockGroupSoftware/concerto_public_api_documentation

RUNNING AN ACTION

Actions are executed by calling `POST /api/action/v1/action/:action_id/invoke`, where ID is the GUID of the `action` to execute.

The body of the request should be a JSON payload which contains a prop for each input a value is needed for.

EXAMPLE FLOW: EXECUTE AN ACTION

We want to execute an `ACCEPT` action against an order with ID 123

1. See what actions are available against order ID 123 with `GET /api/action/v1/action/discover/?q=123`
2. A collection of actions will be returned. One will have a Key of '`ACCEPT`' and an ID of '`XYZ`'
3. See what inputs are required for the `ACCEPT` action with `GET /api/action/v1/action/XYZ/describe/?q=123`
4. A collection of inputs for the action is returned. The `order_id` input is marked as `required`.
5. A Payload is created which includes the `order_id` property and the value `123`
6. The Payload is sent via `POST /api/action/v1/action/XYZ/invoke` to invoke the action. A `HTTP 200` is returned.

…a second or two later…

7. An `OrderAccepted` event is generated because of the changes made by the Action.

INPUTS AND PAYLOADS

When you Invoke an action, you need to provide a payload in the request body containing values for any inputs that the action is expecting. The Actions API expects a JSON object to be provided in the request body with a property for each of these inputs.

AN EXAMPLE

If you `describe` an action and receive back inputs of type `orderId` and `orderPercentComplete` you will need to construct a JSON object like this in your `Invoke` request:

```
{
  “orderId” : “ABC123”,
  “orderPercentComplete” : 51.95
}
```

How do we know how to format the data for the various input types? Refer to the list below for details

INPUT TYPES

The Actions API supports several different input types as defined by the ‘type’ property returned when inspecting an action against an order.

Values for these inputs should be provided in the payload sent to the `POST /api/action/v1/action/XYZ/invoke` endpoint as JSON properties.

It should be expected that the configuration of specific actions may change over time, and that new input types will be introduced from time-to-time. Any integration should be designed in such a way as to allow the inclusion of supported input types on existing actions, or the inclusion of new types of input without having to re-engineer the integration from scratch later.

The following table lists key input types currently supported by the Actions API.

Not every input will be present on every action — the provider only describes the inputs applicable to the specific action being invoked.

RECOMMENDED INPUT TYPE SUPPORT

You don’t need to support every single input type – many of them will be unapplicable for your workflow. Given that you’ll mostly be dealing with Orders, you should focus on supporting the common input types you’ll see when running Order actions. Any that we recommend you support are marked with a ⭐ in the Appendix.

See the appendix for a full list of all input types.

LIST OF INPUT TYPES

KEY RELATIONSHIPS

Relationships between objects returned by the API are expressed as Links. The table below describes the key relationships you’ll find in the Links collection of each object.

API	Object	Link	Details
Budgets V1	Budget Category	Site	
Contract V1	Contract	Client	Optional
		Supplier	
Equipment V1	Equipment Item	Block	
		Area	Optional
		Equipment Sub-type	
	Equipment Sub-type	Equipment Type	
	Equipment Type	Portfolio	
	Equipment Attachment	Content	
		Source	An Equipment Item
Helpdesk V2	Helpdesk Job	Site	
		Block	Optional
		Area	Optional
		Client	
		Equipment Sub-type	Optional

		Supplier	
		Status	
		Trade Affected Status	Optional
		Helpdesk Category	Optional
		Response Category	
		Classification	Optional
		Source Quote	Optional
	Helpdesk Job Status	Portfolio	
		Helpdesk Type	Links to Helpdesk Type. Can be multiple.
	Helpdesk Category	Portfolio	
	Response Category	Order Priority	
		Client	
		Supplier	Optional
	Classification	Response Category	Optional
		Helpdesk Type	Optional
	Helpdesk Log	Helpdesk Job	
		Helpdesk Action	Describes the action that led to the log entry
	Helpdesk Attachment	Content	
		Source	A Helpdesk Job
Location V1	Site	Site Status	
		Client	
	Block	Site	
	Floor	Block	
	Area	Floor	
Orders V1	Order	Cancellation Reason	Optional
		Priority	
		Client	
		Site	
		Contract	
		'Delivered By' Supplier	Optional
		Status	
		Supplier Order Status	Optional
		Helpdesk	Optional
		Input By User	
		Equipment Item	Optional
		Site Logbook	Optional
		Block	Optional
		Area	Optional
		Floor Level	Optional
		Order Type	
		Project	
	Priority	Portfolio	
		Client	
		Supplier	
	Supplier Order Status	Default for Order Status	Optional
	Order Type	Portfolio	
		Project	
		Budget Category	Optional
		Budget Heading	Optional
PPM V1	Site Logbook Action	Order	The Order that fulfils the action
		Site Logbook	
	Site Logbook	Supplier	
		Site	
		Site Logbook Type	
	Site Logbook Type	Portfolio	
		Equipment Sub-Type	
	Site Logbook Action Attachment	Content	

		Source	A Site Logbook Action Job
Quotes V2	Quote	Supplier	
		Quote Status	
		Quote Request	
	Quote Request	Site	
		Source Helpdesk Job	Optional
		Quote Request Status	
		Quote Priority	
		Assigned To User	
		Water Work	Option. This is the source Remedial Request
		Equipment Sub-type	Optional

OPENAPI SPECS

Accompanying this document are several OpenAPI specs covering many of the endpoints and models described above. These specs are based on an in-development snapshot of the API project and do not represent the final form or function of the APIs. They are provided for informational purposes to give an indication of approach.

The latest specs can be obtained from an any instance of concerto from the following URLs:

API		Spec
Actions V1	Execute Actions against Orders, Helpdesk Jobs or Quotes	/api/spec/action-v1/
Budget V1	Get details on Project Budgets	/api/spec/budget-v1/
Contract V1	Find Contracts	/api/spec/contract-v1/
Equipment V1	Get details on Equipment Types and physical equipment items	/api/spec/equipment-v1/
Helpdesk V2	Get details on Helpdesk Jobs and associated items	/api/spec/helpdesk-v2/
Location V1	Find details on Sites, Blocks, Floors and Areas	/api/spec/location-v1/
Orders V1	Get details on Orders (and create Consumable Orders)	/api/spec/orders-v1/
PPM V1	Find information on Site PPMs and PPM Types	/api/spec/ppm-v1/
Quotes V2	Get details on Quote Requests and Quotes	/api/spec/quotes-v2/

RECOMMENDED APPROACHES

This section outlines our recommended principles and approach to integrating with the Concerto API. These examples assume that you are using the recommended Hybrid approach as described above.

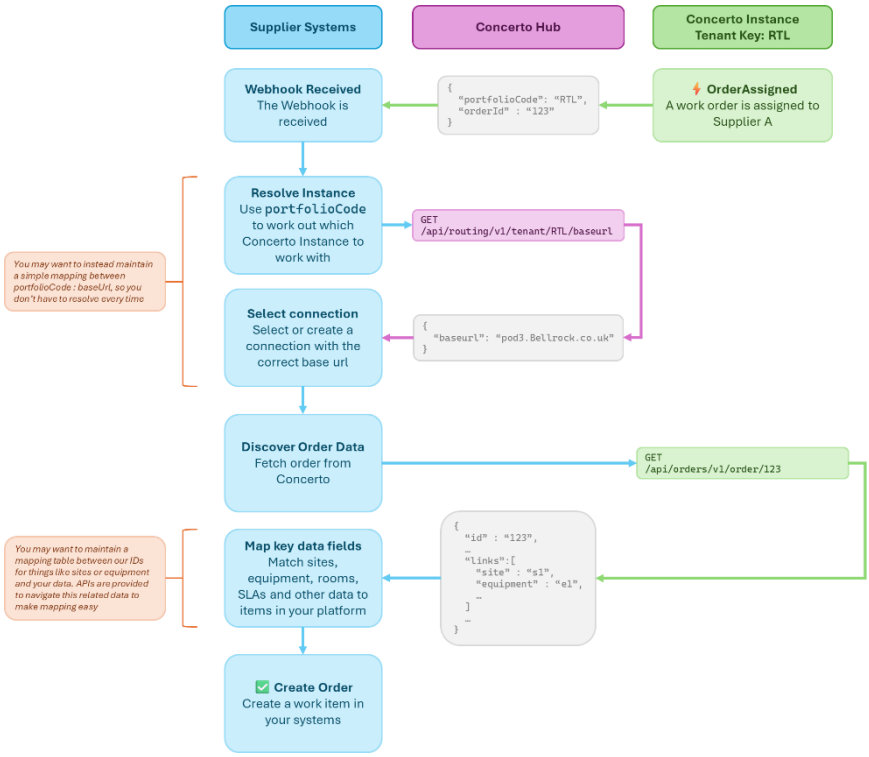
RECEIVE NEW WORK ORDERS FROM CONCERTO

A common integration scenario for a supplier is to receive new work orders form Concerto so they can be created and actioned within their systems. The best way to achieve this is to use `orderAssigned` events to trigger a workflow on your side to fetch details about the work order and create a corresponding item in your platform.

When an `orderAssigned` webhook is delivered to your integration, it will contain an Order ID and a Tenant Key (in the portfolioCode prop).

You should use `portfolioCode` to decide which instance of Concerto you should make your subsequent API calls to.

Mapping a Tenant Key to an API base URL can be achieved by providing the Tenant Key to the 'baseurl' endpoint of the Hub Routing API.

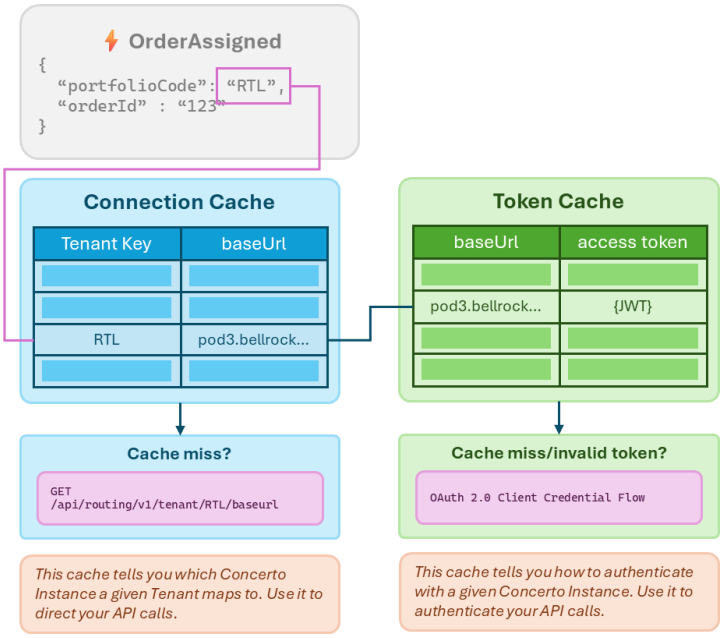


RECOMMENDATION: IMPLEMENT A TWO LEVEL CACHE

You should implement two caches – one to map from a Tenant Key to a Concerto Instance base URL, and another to map from a Concerto Instance base URL to an Access Token. You can use relatively short TTLs on these caches (an hour or so is a good starting point).

It is better to maintain a two level cache as multiple Tenant Keys may map to a single Concerto Base URL, which will all share one access token. In this way you won't be maintaining many access tokens for the same instance.

There is a limit to the number of active access tokens a given user can have against a single instance. Adopting this approach will ensure you stay within that limit.



RECOMMENDATION: KEEP TRACK OF THE ID AND TENANT KEY

When you create an item in your platform to represent a Work Order in Concerto, you should store both the Order ID and the Tenant Key – as you'll need the Tenant Key so you know which Concerto Instance to direct your requests at.

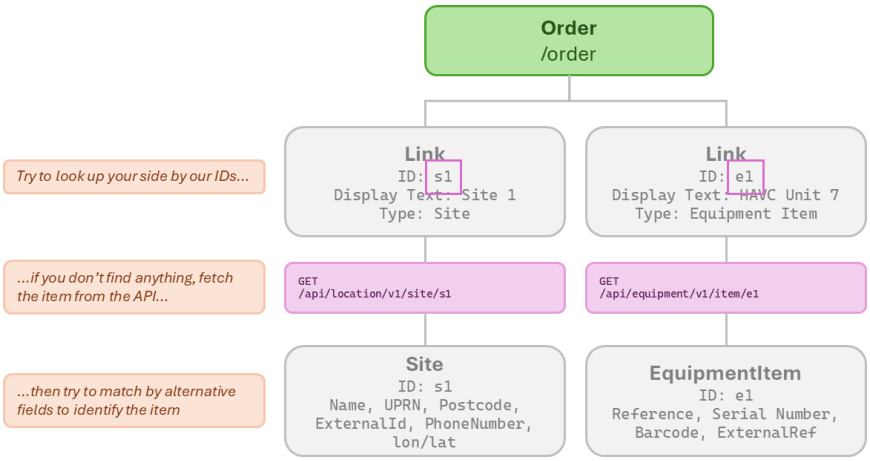
Only have one 'external id' or 'client ref' field available? Store the full order ref URL (`https://{{base_url}}/api/orders/v1/order/:id`) then parse the two key items out of the URL later on by looking at the Authority and the path segments.

RECOMMENDATION: FOLLOW A 'GRAPH TRAVERSAL' APPROACH TO MAP WORKS ORDER DATA

Each item you retrieve from the Concerto API has a 'links' collection, and a works order is no exception. A Link consists of an ID, a Type, a Description and a URL. In the same way you store the Concerto Order ID against items you create, you should also consider how best to map from key items such as a Site or Equipment item to equivalent objects in your platform.

Using the ID is a good choice for this – they are guaranteed to be unique for things like orders, quotes, sites, helpdesk jobs and the like.

You should implement a 'discovery' mechanism to map items with a degree of autonomy when you encounter reference data you have not seen before. How you achieve this mapping is down to the type of data in question – here are two concrete examples:



This is especially important for items such as Sites, where churn in the data may be higher than expected.

RECOMMENDATION: TREAT MAPPING ISSUES AS BAU

You should build mechanisms to clearly show mapping failures and allow users to correct mapping issues, then retry. Reference data often changes as sites open or close, or equipment is added or removed from a site. It's often a good idea to use the data you receive via the Concerto API to keep your own client reference data up to date.

RUN ACTIONS IN CONCERTO

Once you have Orders or other items flowing from Concerto into your platform, the next logical step is automate some of the actions you currently take in the Supplier Portal.

Actions are driven by configuration and may be unique to a given client. Therefore it's recommended that you make your integration somewhat dynamic.

You should maintain some sort of static map between things that occur in your platform, and the corresponding action to run in the context of a particular client. It's important to scope these actions by client, because you may have a different set of actions available depending on who owns the Works Order you're interacting with.

Although the configuration of these actions *can* be dynamic the reality is that most clients follow a very similar workflow, so it's generally safe to match a potential action by name in an order of precedence manner.

Action Map

Tenant Key	Your Event	Action
RTL	PO Received	Accept
LAG	PO Received	Acknowledge

VS

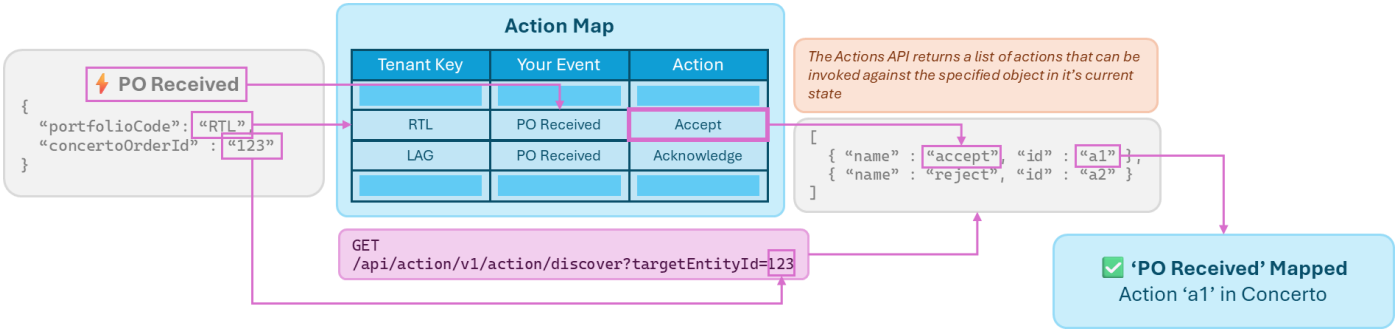
Action Map

Your Event	Action
PO Received	Accept
	Acknowledge

⚠️ **Note:** Whilst Actions IDs may appear to be consistent across clients, you **must not** rely on this. The Action configuration in Concerto is based on a template we deploy when creating instances and may diverge. Match actions by name instead.

RECOMMENDATION: ALWAYS `DISCOVER` ACTIONS BEFORE YOU TRY AND INVOKE THEM

Because Actions in Concerto have quite complex scoping behaviours, it's always best to check which actions are available for a given item before trying to invoke them. Use the Actions API and your mapping table to resolve which Action should be invoked



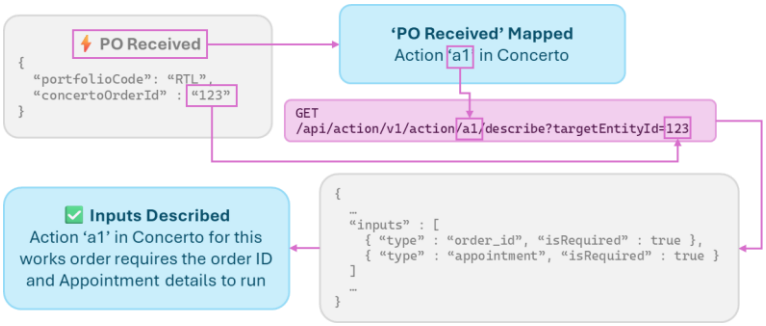
Unsure which action applies in which circumstance? It can be complex if you work across many clients in multiple sectors. The Actions you receive from the API are the same actions you see in the Supplier Portal, so consult your supplier pack for details of exactly which Action you should be invoking.

RECOMMENDATION: ALWAYS `DESCRIBE` ACTIONS BEFORE YOU TRY AND INVOKE THEM

In the same way that the name of the specific action you might want to invoke may vary, so too might the inputs that an Action requires in order to run.

For example, some clients may require that you provide an ETA via an 'appointment' input when accepting an order, whilst others may not need that.

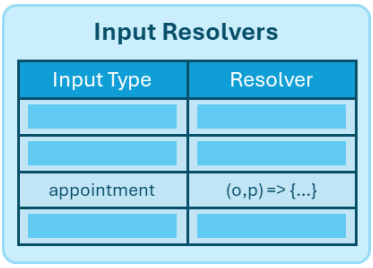
For this reason, after mapping to an action, you should always ask the Actions API to describe required inputs.



RECOMMENDATION: ADOPT A GENERIC APPROACH TO PROVIDING ACTION INPUTS

Because of the dynamic nature of action inputs, instead of hardcoding a mapping between a given event in your platform, an action and it's required inputs, you should instead adopt an approach whereby you have a table of 'resolvers' for each type of input returned by the `describe` endpoint which is capable of adding properties to the Actions API invoke payload.

This means that if the Action configuration changes and new inputs are required, your integration will not break.



APPENDIX A: INPUT TYPES

⭐ OrderId 16
⭐ Appointment.From 16
⭐ Appointment.To 17
⭐ Appointment.Notes 17
⭐ Appointment.CommunicationMethodId 17
⭐ Base64Files 17
ClientReference 18
ContractJobDetail.ActionId 18
ContractJobDetail.Title 18
ContractJobDetail.Description 19
ContractJobDetail.EventDate 19
ContractJobDetail.ContractJobEstimates[] 19
ContractJobDetail.LabourCost 20
ContractJobDetail.MaterialCost 20
ContractJobDetail.PlantCost 20
ContractJobDetail.OtherCost 20
ContractJobDetail.NrEngineers 21
ContractJobDetail.EstimateHours 21
CreatePPM 21
DocSlotFiles 21
⭐ EstimatedArrivalTime 22
ExistingMarkedAsPPMCertificateDocumentIds 22
HelpdeskCauseId 22
HelpdeskFlagForFurtherAttention 22
⭐ IsNoCharge 23
⭐ NoChargeNote 23
⭐ Notes 23
Operatives 23
⭐ OrderActionFieldInputs 23
⭐ OrderActualResponseDate 24
⭐ OrderCompleteDate 24
⭐ OrderPercentComplete 24
OrderSiteLogbookActionId 25
QuoteRequestTitle 25
QuoteRequest.Description 25
QuoteRequest.QuotePriorityId 25
QuoteRequest.HelpdeskCategoryId 26
⭐ SupplierOrderReference 27
⭐ supplier_order_status_id 27
⭐ supplier_order_status_comment 28
⭐ Timings 28
⭐ TimeExtensionRequest.TargetResponse 28
⭐ TimeExtensionRequest.TargetComplete 28
OrderIds 29
OrderId 29
DocumentIds 29
EmailAddresses 29
GoodsReceivedId 30
GoodsReceivedDate 30
LineItems 30
LoginName 31
Message 31
Notes 31
Password 31
ReceivedMethod 31
Reference 32
ShouldCancelOrder 32
ShouldIncludeOrderValueInEmail 32

WorkflowStageId 32
helpdesk_id 33
file_name 33
file_content_base64 33
⭐ QuoteRequestId 33
AssignUserId 33
AutoAssignCapexCode 34
⭐ DescriptionForOrder 34
IsRechargeText 34
⭐ Message 34
OrderBudgetCategoryApportionments 35
OrderBudgetHeadingId 35
OrderHelpdeskCategoryId 35
OrderHelpdeskTypeId 35
OrderPriorityId 36
OrderProjectId 36
⭐ QuoteCostCode 36
⭐ QuoteFieldAnswers 36
OrderBudgetCategoryId 37
QuoteBusinessUnitId 37
OrderCapexCodeId 37
OrderCostCode 37
OrderCustomerRef 38
RejectLinks 38
SectionsAccepted 38
SuccessfulQuoteId 38
⭐ SupplierIds 39
UpdateWaterWorkStatusId 39
UserIds 39
ChangeRequestId 40
Message 40

ORDER ACTIONS

This section covers all action inputs applicable to Order entities.

⭐ OrderId

The unique identifier of the order the action is being performed against.

Payload property: `OrderId`

```
{
  "OrderId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: None.

⭐ Appointment.From

The start date and time of the appointment being booked. Present when the action is an appointment action.

Payload property: `Appointment.From`

```
{
  "Appointment": {
    "From": "2024-06-01T09:00:00Z"
  }
}
```

Metadata: `DateMeta`

Appointment.To

The end date and time of the appointment being booked. Present when the action is an appointment action.

Payload property: `Appointment.To`

```
{
  "Appointment": {
    "To": "2024-06-01T11:00:00Z"
  }
}
```

Metadata: `DateMeta`

Appointment.Notes

Optional free-text notes to accompany the appointment booking.

Payload property: `Appointment.Notes`

```
{
  "Appointment": {
    "Notes": "Please call ahead on arrival."
  }
}
```

Metadata: None.

Appointment.CommunicationMethodId

The communication method used to confirm the appointment (e.g. phone, email). Present when the action is an appointment action and communication methods are configured in the system.

Payload property: `Appointment.CommunicationMethodId`

```
{
  "Appointment": {
    "CommunicationMethodId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  }
}
```

Metadata: `OptionSelectMeta<Guid>`

Base64Files

Files to attach to the order or PPM visit record, provided as base64-encoded data. Present when the action is configured to accept file uploads (`is_files`). May be required depending on the action configuration (e.g. PPM statutory orders, orders with remedials, or when `upload_status` is set to mandatory for all).

Payload property: `Base64Files`

```
{
  "Base64Files": [
    {
      "Name": "job-sheet.pdf",
      "Data": "JVBERi0xLjQKJeLjz9MKMSAwIG9iago..."
    }
  ]
}
```

Each entry:

Existing server-side files can also be referenced via `ExistingFiles`:

```
{
  "ExistingFiles": [
    {
      "NameOnDiskWithExtension": "abc123def456.pdf"
    }
  ]
}
```

Metadata: None.

ClientReference

The client's own reference string for the order (e.g. a purchase order number or works reference). Present when the action is configured to prompt for a client reference.

Payload property: `ClientReference`

```
{
  "ClientReference": "PO-2024-00123"
}
```

Metadata: `StringMeta`

ContractJobDetail.ActionId

The contract job action type to record against this contract notice. Present when the order action is linked to a contract job type.

Payload property: `ContractJobDetail.ActionId`

```
{
  "ContractJobDetail": {
    "ActionId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  }
}
```

Metadata: `OptionSelectMeta<Guid>`

ContractJobDetail.Title

A short descriptive title for the contract notice. Present when the order action is linked to a contract job type.

Payload property: `ContractJobDetail.Title`

```
{
  "ContractJobDetail": {
    "Title": "Emergency repair to flat roof"
  }
}
```

Metadata: None.

ContractJobDetail.Description

A longer description of the contract notice work. Present when the order action is linked to a contract job type.

Payload property: `ContractJobDetail.Description`

```
{
  "ContractJobDetail": {
    "Description": "Repair delamination and apply new waterproofing membrane."
  }
}
```

Metadata: None.

ContractJobDetail.EventDate

The date and time the contract job event took place. The label for this date is configured per contract job type. Present when the order action is linked to a contract job type.

Payload property: `ContractJobDetail.EventDate`

```
{
  "ContractJobDetail": {
    "EventDate": "2024-06-01T10:00:00Z"
  }
}
```

Metadata: None.

ContractJobDetail.ContractJobEstimates[]

Line-item cost estimates for the contract notice, drawn from the contract schedule of rates. Present when the order action is linked to a non-simple-estimate contract job type.

Payload property: `ContractJobDetail.ContractJobEstimates`

```
{
  "ContractJobDetail": {
    "ContractJobEstimates": [
      {
        "ScheduleId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
        "Quantity": 2.5,
        "Description": "Supply and fix slate tiles",
        "Rate": 45.00,
        "Markup": 10.0
      }
    ]
  }
}
```

Metadata: `CustomFieldMeta` (optional) — if the contract job type has custom fields attached to estimates, the metadata will include:

Each `CustomField`:

ContractJobDetail.LabourCost

Labour cost for the job. Present when the order action is linked to a **simple estimate** contract job type.

Payload property: `ContractJobDetail.LabourCost`

```
{
  "ContractJobDetail": {
    "LabourCost": 120.00
  }
}
```

Metadata: None.

ContractJobDetail.MaterialCost

Material cost for the job. Present when the order action is linked to a **simple estimate** contract job type.

Payload property: `ContractJobDetail.MaterialCost`

```
{
  "ContractJobDetail": {
    "MaterialCost": 75.50
  }
}
```

Metadata: None.

ContractJobDetail.PlantCost

Plant/equipment hire cost for the job. Present when the order action is linked to a **simple estimate** contract job type.

Payload property: `ContractJobDetail.PlantCost`

```
{
  "ContractJobDetail": {
    "PlantCost": 0.00
  }
}
```

Metadata: None.

ContractJobDetail.OtherCost

Ad-hoc / miscellaneous cost for the job. Present when the order action is linked to a **simple estimate** contract job type.

Payload property: `ContractJobDetail.OtherCost`

```
{
  "ContractJobDetail": {
    "OtherCost": 15.00
  }
}
```

Metadata: None.

ContractJobDetail.NrEngineers

The number of engineers required for the job. Present when the order action is linked to a **simple estimate** contract job type.

Payload property: `ContractJobDetail.NrEngineers`

```
{
  "ContractJobDetail": {
    "NrEngineers": 2
  }
}
```

Metadata: None.

ContractJobDetail.EstimateHours

The estimated labour hours for the job. Present when the order action is linked to a **simple estimate** contract job type.

Payload property: `ContractJobDetail.EstimateHours`

```
{
  "ContractJobDetail": {
    "EstimateHours": 4.0
  }
}
```

Metadata: None.

CreatePPM

Controls whether to link the visit to an existing PPM schedule record or create a new one. Present when the action is linked to a PPM sitelogbook status and no PPM visit record has yet been created for the order, and multiple existing PPM activities are available to link to.

Payload property: `CreatePPM`

```
{
  "CreatePPM": 0
}
```

Values: `0` = link to an existing PPM record (specify `OrderSiteLogbookActionId`); `1` = create a new PPM record.

Metadata: `OptionSelectMeta<int>`

DocSlotFiles

Files to upload into specific named document slots defined for the action. One `DocSlotFiles` input is described per required document slot. Present when the action has document slots configured with a `FormEntry` rule.

Payload property: `DocSlotFiles`

```
{
  "DocSlotFiles": [
    {
      "DocSlotId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "InstanceId": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
      "FileName": "certificate.pdf",
      "OldFileName": "cert_on_disk_abc123.pdf",
      "FileType": "pdf",
      "StartDate": "2024-06-01T00:00:00Z"
```

```
    }
  ]
}
```

Each entry:

Metadata: `DocSlotSpec` — describes the document slot being fulfilled:

EstimatedArrivalTime

The ETA for the supplier to attend the Order

Payload property: `EstimatedArrivalTime`

```
{
  "EstimatedArrivalTime": "2024-06-01T00:00:00Z"
}
```

Metadata: `DateMeta`

ExistingMarkedAsPPMCertificateDocumentIds

The IDs of previously uploaded documents that should be marked as the PPM certificate(s) for this visit. Present when the action is a PPM action with certificate selection enabled (`is_ppm_cert_select`), the order has an existing PPM visit record, and documents have already been uploaded.

Payload property: `ExistingMarkedAsPPMCertificateDocumentIds`

```
{
  "ExistingMarkedAsPPMCertificateDocumentIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ]
}
```

Metadata: `OptionMultiMeta<Guid>`

HelpdeskCauseId

The root cause of the helpdesk job (e.g. wear and tear, accidental damage). Present when the action is configured to capture a root cause, the order has an associated helpdesk job, and multiple causes are available.

Payload property: `HelpdeskCauseId`

```
{
  "HelpdeskCauseId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: `OptionSelectMeta<Guid>`

HelpdeskFlagForFurtherAttention

Whether the helpdesk job should be flagged for further attention. Present when the action is a flag action, the order has an associated helpdesk job, and the invoking user does not have the `supplier_portal_cannot_flag` role.

Payload property: `HelpdeskIsFlagged`

```
{
  "HelpdeskIsFlagged": true
}
```

Metadata: None.

IsNoCharge

Indicates whether the order should be closed with no charge to the client. Present when the action is configured to support no-charge closure (`no_charge` is not hidden).

Payload property: `IsNoCharge`

```
{
  "IsNoCharge": true
}
```

Metadata: None.

NoChargeNote

The reason why there is no charge for the order. Submitted alongside `IsNoCharge`. Present when the action is configured to support no-charge closure.

Payload property: `NoChargeNote`

```
{
  "NoChargeNote": "Fault found to be pre-existing, outside of contract scope."
}
```

Metadata: None.

Notes

General free-text notes to record against the action. Present when the action is configured to capture notes (`notes` is not hidden).

Payload property: `Notes`

```
{
  "Notes": "Work carried out as requested. Minor additional damage noted around window frame."
}
```

Metadata:

Operatives

The IDs of operatives (field engineers / workers) to assign to this order. Present when the action is configured to assign operatives.

Payload property: `Operatives`

```
{
  "Operatives": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "6ba7b810-9dad-11d1-80b4-00c04fd430c8"
  ]
}
```

Metadata: `OptionMultiMeta<Guid>`

OrderActionFieldInputs

Custom field answers for the action. Present when the action has custom fields configured (at least one custom field is available).

Payload property: `OrderActionFieldInputs`

```
{
  "OrderActionFieldInputs": [
    {
      "CustomFieldId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "Input": "Some text answer"
    },
    {
      "CustomFieldId": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
      "Input": 42
    }
  ]
}
```

Metadata: `CustomFieldMeta`

Each `CustomField`:

OrderActualResponseDate

The actual date and time the engineer attended / responded to the order. Present when the action is configured to capture a response date (`response_time` is not hidden).

Payload property: `OrderActualResponseDate`

```
{
  "OrderActualResponseDate": "2024-06-01T09:15:00Z"
}
```

Metadata: `DateMeta`

OrderCompleteDate

The date and time the work on the order was completed. Present when the action is configured to capture a completion date (`complete_time` is not hidden).

Payload property: `OrderCompleteDate`

```
{
  "OrderCompleteDate": "2024-06-01T14:30:00Z"
}
```

Metadata: `DateMeta`

OrderPercentComplete

The percentage of work completed on the order so far. Present when the action is configured as a percent-complete action.

Payload property: `OrderPercentComplete`

```
{
  "OrderPercentComplete": 75
}
```

Metadata:

ORDERQUANTITIES

Quantity delivered/received for each line item on an equipment order. Present when the order is an equipment order (`is_equipment_order`).

Payload property: `OrderQuantities`

```
{
  "OrderQuantities": [
    {
      "OrderSplitId": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "Quantity": 10.0
    }
  ]
}
```

Each entry:

Metadata: `OrderQuantity`

OrderSiteLogbookActionId

The ID of the existing PPM schedule activity to link this order visit to. Present alongside `CreatePPM` when there are existing PPM activities available. Only required when `CreatePPM` is `0`.

Payload property: `OrderSiteLogbookActionId`

```
{
  "OrderSiteLogbookActionId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: `OptionSelectMeta<Guid>`

QuoteRequestTitle

A short title for the new quote request being raised from the order. Present when the action is a quote-request action.

Payload property: `QuoteRequest.Title`

```
{
  "QuoteRequest": {
    "Title": "Replacement boiler — Block A"
  }
}
```

Metadata: `StringMeta`

QuoteRequest.Description

A detailed description of the work to be quoted. Present when the action is a quote-request action.

Payload property: `QuoteRequest.Description`

```
{
  "QuoteRequest": {
    "Description": "Supply and install a new A-rated combination boiler."
  }
}
```

Metadata: None.

QuoteRequest.QuotePriorityId

The priority of the quote request. Present when the action is a quote-request action.

Payload property: `QuoteRequest.QuotePriorityId`

```
{
  "QuoteRequest": {
    "QuotePriorityId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  }
}
```

Metadata: `OptionSelectMeta<Guid>`

QuoteRequest.HelpdeskCategoryId

The helpdesk category to associate with the quote request. Present when the action is a quote-request action and multiple categories exist for the site.

Payload property: `QuoteRequest.HelpdeskCategoryId`

```
{
  "QuoteRequest": {
    "HelpdeskCategoryId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  }
}
```

Metadata: `OptionSelectMeta<Guid>`

★ QUOTE ID

The ID of a quote to execute an action against

Payload property: `quote_id`

```
{
 "quote_id": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

★ REMEDIALS

New or updated remedial records to record against this PPM visit. Present when the action is configured to capture remedial work (`is_remedials`).

Payload property: `Remedials`

```
{
  "Remedials": [
    {
      "Id": "3fa85f64-5717-4562-b3fc-2c963f66afa6",
      "PriorityId": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
      "ActionRequired": "Replace corroded pipework in roof void",
      "Cost": 450.00,
      "EquipmentId": null
    }
  ]
}
```

Each entry:

Metadata: `RemedialMeta`

REMEDIALCARRYFORWARDIDS

IDs of outstanding remedial records from previous PPM visits that should be carried forward and re-linked to this visit. Present when the action involves remedials, the invoking user has the `can_forward_remedials` role, and there are outstanding remedials available to carry forward.

Payload property: `RemedialCarryForwardIds`

```
{
  "RemedialCarryForwardIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ]
}
```

Metadata: `RemedialCarryForwardMeta`

Each `Remedial` in `AvailableToCarryForward`:

★ SupplierOrderReference

The supplier's own reference number for this order (e.g. their internal job number). Present when the action is configured to capture a supplier reference (`supplier_reference` is not hidden).

Payload property: `SupplierOrderReference`

```
{
  "SupplierOrderReference": "SUP-JOB-98765"
}
```

Metadata: `StringMeta`

SITELOGBOOKEQUIPMENTACTIVITYSTATUS

Updated activity statuses for each piece of equipment associated with the PPM visit. Present when the action is configured to update equipment status (`is_update_equipment`), the order is linked to a sitelogbook, and equipment records exist for the sitelogbook.

Payload property: `SitelogbookEquipmentActivityStatus`

A dictionary mapping each `equipmentId` (Guid) to a chosen `equipmentActivityStatusId` (Guid):

```
{
  "SitelogbookEquipmentActivityStatus": {
    "3fa85f64-5717-4562-b3fc-2c963f66afa6": "6ba7b810-9dad-11d1-80b4-00c04fd430c8",
    "7c9e6679-7425-40de-944b-e07fc1f90ae7": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  }
}
```

Metadata: `EquipmentStatusMeta`

★ supplier_order_status_id

The unique identifier of the supplier order status to apply. Must correspond to an existing status returned by `ISupplierOrderStatusRepository.All()`.

Payload property: `supplier_order_status_id`

```
{
  "supplier_order_status_id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890"
}
```

Metadata: Raw `JObject` with a single `options` array populated at describe-time from all available supplier order statuses.

supplier_order_status_comment

An optional free-text comment to accompany the status change. Stored against the order and recorded in the status change history.

Payload property: `supplier_order_status_comment`

```
{
  "supplier_order_status_comment": "Parts have been dispatched by the supplier."
}
```

Metadata: None.

Timings

Multiple attendance time slots for the job (e.g. where an engineer attended more than once). Present when the action is a multi-attendance action.

Payload property: `Timings`

```
{
  "Timings": [
    {
      "Start": "2024-06-01T09:00:00Z",
      "End": "2024-06-01T12:00:00Z",
      "IsSecondOp": false,
      "Description": "Initial attendance"
    },
    {
      "Start": "2024-06-02T14:00:00Z",
      "End": "2024-06-02T16:30:00Z",
      "IsSecondOp": true,
      "Description": "Return visit"
    }
  ]
}
```

Each timing entry:

Metadata: None.

TimeExtensionRequest.TargetResponse

The new requested target response date for the order (time extension). Present when the action is a time-extension action.

Payload property: `TimeExtensionRequest.TargetResponse`

```
{
  "TimeExtensionRequest": {
    "TargetResponse": "2024-06-15T12:00:00Z"
  }
}
```

Metadata: None.

TimeExtensionRequest.TargetComplete

The new requested target completion date for the order (time extension). Present when the action is a time-extension action.

Payload property: `TimeExtensionRequest.TargetComplete`

```
{
  "TimeExtensionRequest": {
```

```
    "TargetComplete": "2024-07-01T12:00:00Z"
  }
}
```

Metadata: None.

CONSUMABLE ORDER ACTIONS

This section covers all action inputs applicable to Consumable Order entities.

OrderIds

The unique identifiers of the consumable orders for which approval is being requested. One or more order GUIDs must be supplied; validation fails if the array is null or empty.

Payload property: `OrderIds`

```
{
  "OrderIds": [
    "75bdea1a-97ef-439c-b21d-e58683e1c3ec",
    "3fa85f64-5717-4562-b3fc-2c963f66afa6"
  ]
}
```

Metadata: None.

OrderId

The unique identifier of the consumable order a GRN or rejection is being recorded against.

Payload property: `OrderId`

```
{
  "OrderId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: None.

DocumentIds

GUIDs of documents to attach to the orders before approval. Omitting or supplying an empty array results in no document attachments.

Payload property: `DocumentIds`

```
{
  "DocumentIds": [
    "a1b2c3d4-0000-0000-0000-000000000001"
  ]
}
```

Metadata: None.

EmailAddresses

The email addresses to send the approval notification to. If empty, falls back to the supplier's own email addresses from the order record.

Payload property: `EmailAddresses`

```
{
  "EmailAddresses": ["supplier@example.com", "manager@example.com"]
```

```
}
```

Metadata: None.

GoodsReceivedId

The ID of an existing GRN to update. When provided and non-empty, `Invoke` loads and edits the existing GRN rather than creating a new one. Omit or pass `null` / `Guid.Empty` to create a new GRN.

Payload property: `GoodsReceivedId`

```
{
  "GoodsReceivedId": "7c9e6679-7425-40de-944b-e07fc1f90ae7"
}
```

Metadata: None.

GoodsReceivedDate

The date on which the goods were received.

Payload property: `GoodsReceivedDate`

```
{
  "GoodsReceivedDate": "2024-06-01T00:00:00Z"
}
```

Metadata: None.

LineItems

An array of per-split GRN entries. Each element corresponds to one order split line and captures the delivery status, quantity received (when `ReceivedMethod` is `ByQuantity`), or net value received (when `ReceivedMethod` is `ByValue`).

Each element maps to `GoodsReceivedLineItem`:

Payload property: `LineItems`

```
{
  "LineItems": [
    {
      "Id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
      "SplitOrderStatusId": "d4e5f6a7-b8c9-0123-def4-567890abcdef",
      "QuantityReceived": 5.0,
      "NetValue": 0.0,
      "UnitValue": 0.0,
      "QuantityOrdered": 10.0
    }
  ]
}
```

Metadata: Custom `JObject` built inline in `Describe()`.

```
{
  "statusValues": [
    { "id": "d4e5f6a7-b8c9-0123-def4-567890abcdef", "name": "Fully Delivered" },
    { "id": "e5f6a7b8-c9d0-1234-ef56-7890abcdef01", "name": "Partly Delivered" }
  ],
  "items": [
    {
```

```
      "id": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
      "unit_rate": 150.00,
      "quantity_ordered": 10.0
    }
  ]
}
```

LoginName

The approving user's login name. **Only required when the user does not have SSO auto-authentication enabled.**

Payload property: `LoginName`

```
{
  "LoginName": "jsmith"
}
```

Metadata: None.

Message

A free-text message to accompany the approval request notification. When omitted or empty, the default message `"Please approve the following order : "` is used.

Payload property: `Message`

```
{
  "Message": "Please review and approve this consumable order at your earliest convenience."
}
```

Metadata: None.

Notes

Free-text notes explaining why the order is being rejected.

Payload property: `Notes`

```
{
  "Notes": "Duplicate request — original order P-00123 already approved."
}
```

Metadata: None.

Password

The approving user's password, paired with `LoginName`. **Only required when SSO auto-authentication is not enabled.**

Payload property: `Password`

```
{
  "Password": "s3cr3t!"
}
```

Metadata: None.

ReceivedMethod

Determines whether the GRN is recorded by quantity delivered or by net value delivered. Maps to the `ReceivedMethod` enum.

Payload property: `ReceivedMethod`

```
{
  "ReceivedMethod": 0
```

```
}
```

Metadata: `OptionSelectMeta<ReceivedMethod>`

Reference

An optional free-text reference string to record against the GRN (e.g. a delivery note number or supplier reference).

Payload property: `Reference`

```
{
  "Reference": "DN-2024-00456"
}
```

Metadata: None.

ShouldCancelOrder

Boolean flag indicating whether the underlying consumable order should also be cancelled as part of the rejection.

Payload property: `ShouldCancelOrder`

```
{
  "ShouldCancelOrder": true
}
```

Metadata: None.

ShouldIncludeOrderValueInEmail

Whether the order value should be included in the body of the approval email.

Payload property: `ShouldIncludeOrderValueInEmail`

```
{
  "ShouldIncludeOrderValueInEmail": false
}
```

Metadata: None.

SHOULDSENDEMAIL

Controls whether an approval email is sent to the supplier(s) after the orders are authorised.

Payload property: `ShouldSendEmail`

```
{
  "ShouldSendEmail": true
}
```

Metadata: None.

WorkflowStageId

The ID of a specific workflow stage to route the approval request to. When provided, `order.RequestWorkflowStage` is called instead of the standard `order.RequestApproval`. Omit to use the default approval path.

Payload property: `WorkflowStageId`

```
{
  "WorkflowStageId": "6ba7b810-9dad-11d1-80b4-00c04fd430c8"
}
```

Metadata: None.

HELPDESK JOB ACTIONS

This section covers all action inputs applicable to Helpdesk Job entities. It encompasses the `AddAttachmentAction` discoverable action (`ADD_ATTACHMENT`).

helpdesk_id

The unique identifier of the Helpdesk Job to attach the file to.

Payload property: `helpdesk_id`

```
{
  "helpdesk_id": "743df5eb-ccb1-451f-9120-ec5788bde9d3"
}
```

Metadata: None.

file_name

The filename (including extension) to assign to the stored attachment.

Payload property: `file_name`

```
{
  "file_name": "site-report.pdf"
}
```

Metadata: None.

file_content_base64

The binary content of the file encoded as a Base64 string.

Payload property: `file_content_base64`

```
{
  "file_content_base64": "JVBERi0xLjQKJ..."
}
```

Metadata: None.

QUOTE REQUEST ACTIONS

This section covers all action inputs applicable to Quote Request entities.

QuoteRequestId

The unique identifier of the quote request the action is being performed against.

Payload property: `QuoteRequestId`

```
{
  "QuoteRequestId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: None.

AssignUserId

The ID of the user to whom the quote request should be assigned. Present when the action is an assign action (`IsAssign`). Always required.

Payload property: `AssignUserId`

```
{
  "AssignUserId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: None.

AutoAssignCapexCode

Whether to automatically assign the next available CAPEX/cost code to the resulting order. Present when the action is a cost-code action (`IsCostCode`), the portfolio limits CAPEX code usage (`limit_capex_code_usage`), the quote request does not already have an assigned CAPEX code, and at least one CAPEX code is available for the portfolio.

Payload property: `AutoAssignCapexCode`

```
{
  "AutoAssignCapexCode": true
}
```

Metadata: None.

DescriptionForOrder

The description text that will appear on the resulting order. Present when the action is an order-details action (`IsOrderDetails`). Always required. Pre-populated with the text `* QUOTE *` followed by the quote request description.

Payload property: `DescriptionForOrder`

```
{
  "DescriptionForOrder": "* QUOTE *\nReplace failed extract fan in kitchen."
}
```

Metadata: `StringMeta`

IsRechargeText

Whether the resulting order should be flagged as rechargeable. Present when the invoking user has the `canChangeRecharge` role and the action is a recharge action (`IsRecharge`). Always required.

Payload property: `IsRechargeText`

```
{
  "IsRechargeText": "1"
}
```

Values: `"0"` = not set / unspecified; `"1"` = rechargeable; `"2"` = not rechargeable.

Metadata: `OptionSelectMeta<int>`

Message

Free-text message body to include in the email notification. Present when the action sends any email — to the assignee (`IsEmailAssignee`), the owner (`IsEmailOwner`), suppliers (`IsEmailSuppliers`), or specific other users (`IsEmailOthers`).

Payload property: `Message`

```
{
  "Message": "Please review and respond to this quote request at your earliest convenience."
}
```

Metadata: None.

OrderBudgetCategoryApportionments

A dictionary of budget category IDs to percentage allocations for apportioning the order cost across multiple budget categories. Present when the action is an order-details action (`IsOrderDetails`) and the portfolio is configured for reactive budget category apportionment (`reactive_budgetcategory_apportionment`).

Payload property: `OrderBudgetCategoryApportionments`

```
{
  "OrderBudgetCategoryApportionments": {
    "3fa85f64-5717-4562-b3fc-2c963f66afa6": 60.0,
    "6ba7b810-9dad-11d1-80b4-00c04fd430c8": 40.0
  }
}
```

Metadata: `BudgetCategoryApportionment`

OrderBudgetHeadingId

The budget heading to assign to the resulting order. Present in two distinct scenarios:

1. **Budget-details action** (`IsBudgetDetails && !IsOrderDetails`): when the portfolio is configured to source the budget heading from the asset sub-type (and the sub-type has a project set) or when the action does not have a fixed budget heading set.
2. **Order-details action** (`IsOrderDetails`): when the portfolio is configured to source the budget heading from the asset sub-type (and the sub-type has a project set), or when the portfolio uses reactive budget category apportionment, or when the budget category label is not shown and no fixed budget heading is configured on the action.

Payload property: `OrderBudgetHeadingId`

```
{
  "OrderBudgetHeadingId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: `OptionSelectMeta<Guid>`

OrderHelpdeskCategoryId

The helpdesk category to assign to the resulting order. Present when the action is an order-details action (`IsOrderDetails`), the quote request is not from the project module, more than one category is configured, and the `QuoteHelpdeskCategory` page-setting label is not hidden.

Payload property: `OrderHelpdeskCategoryId`

```
{
  "OrderHelpdeskCategoryId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: `OptionSelectMeta<Guid>`

OrderHelpdeskTypeId

The helpdesk job type to assign to the resulting order. Present when the action is an order-details action (`IsOrderDetails`), the quote request is not from the project module, no job type is fixed on the action, and more than one job type is configured (i.e. the data table contains more than two rows including the blank row).

Payload property: `OrderHelpdeskTypeId`

```
{
  "OrderHelpdeskTypeId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: `OptionSelectMeta<Guid>`

OrderPriorityId

The priority to assign to the resulting order. Present when the action is an order-details action (`IsOrderDetails`) and priority selection is allowed. Priority selection is disallowed only when the client has `is_allow_supplier_priority` enabled and all related quotes already have an order priority set.

Payload property: `OrderPriorityId`

```
{
  "OrderPriorityId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: `OptionSelectMeta<Guid>`

OrderProjectId

The project to associate with the resulting order. Present when the action is an order-details action (`IsOrderDetails`), the portfolio is not configured to source the project from the asset sub-type, and the action does not have a fixed project pre-configured (`act.OrderProjectId` is not set).

Payload property: `OrderProjectId`

```
{
  "OrderProjectId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: `OptionSelectMeta<Guid>`

QuoteCostCode

The cost code to record against the quote request. Present when the action is an order-details action (`IsOrderDetails`) and the `QuoteCostCode` page-setting label is not hidden. This is an informational/pre-filled field — the current cost code from the quote request is provided as the default. Maximum length is 40 characters.

Payload property: `QuoteCostCode`

```
{
  "QuoteCostCode": "CC-MECH-2024"
}
```

Metadata: `StringMeta`

QuoteFieldAnswers

Answers to the custom fields configured for the quote action. Present when at least one custom field is defined for the action. Whether any individual field is required depends on the field's own `IsMandatory` flag; the `ActionInput.IsRequired` flag on this input is `true` if any field is mandatory.

Payload property: `QuoteFieldAnswers`

```
{
  "QuoteFieldAnswers": {
    "3fa85f64-5717-4562-b3fc-2c963f66afa6": "Some text answer",
```

```
    "6ba7b810-9dad-11d1-80b4-00c04fd430c8": "Option A"
  }
}
```

The object is a dictionary keyed by custom field ID (`Guid`) with string values.

Metadata: `CustomFieldMeta`

Each `CustomField`:

OrderBudgetCategoryId

The budget category to assign to the resulting order. Present in two scenarios:

1. **Budget-details action** (`IsBudgetDetails && !IsOrderDetails`): when the `QuoteRequestBudgetCategory` page-setting label is not hidden.
2. **Order-details action** (`IsOrderDetails`): when the portfolio does not use reactive budget category apportionment and the budget category should be shown (based on workspace configuration for order-details or quote-details paths respectively).

Payload property: `OrderBudgetCategoryId`

```
{
  "OrderBudgetCategoryId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: `OptionSelectMeta<Guid>`

QuoteBusinessUnitId

The business unit to associate with the quote request. Present when the action has `IsBusinessUnit` set, the `ORDER_BUSINESS_UNIT` system setting is enabled, and more than one business unit is configured in the system.

Payload property: `QuoteBusinessUnitId`

```
{
  "QuoteBusinessUnitId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: `OptionSelectMeta<Guid>`

OrderCapexCodeId

The CAPEX/cost code to assign to the resulting order. Present when the action is a cost-code action (`IsCostCode`), the portfolio does not limit CAPEX code usage (i.e. codes are selectable rather than auto-assigned), and at least one CAPEX code is configured for the portfolio.

Payload property: `OrderCapexCodeId`

```
{
  "OrderCapexCodeId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: `OptionSelectMeta<Guid>`

OrderCostCode

A free-text cost code to record against the resulting order. Present when the action is a cost-code action (`IsCostCode`), the portfolio does not limit CAPEX code usage, and no CAPEX codes have been configured (the CAPEX code dropdown would be empty), so a free-text entry is offered instead.

Payload property: `OrderCostCode`

```
{
  "OrderCostCode": "CC-REACTIVE-001"
}
```

Metadata: None.

OrderCustomerRef

The client's own reference for the resulting order. Present when the action is an order-details action (`IsOrderDetails`), the action has `IsCustomerRefPrompt` set, and the `HelpdeskClientRef` page-setting label is not hidden. Whether the field is required is determined by the page-setting's `isMandatory` flag. Maximum length is 40 characters.

Payload property: `OrderCustomerRef`

```
{
  "OrderCustomerRef": "PO-2024-00456"
}
```

Metadata: `StringMeta`

RejectLinks

Whether to also reject any linked quote requests when accepting the successful quote. Present when the action is a successful-quote action (`IsSuccessful`), a rejected action is configured on the action (`RejectedActionId` is set), and the quote request has linked quote requests (`HasLinks()` is true). Always required.

Payload property: `RejectLinks`

```
{
  "RejectLinks": "1"
}
```

Values: `0` = unspecified; `1` = Yes (reject linked quotes); `2` = No.

Metadata: `OptionSelectMeta<int>`

SectionsAccepted

Which sections of a project quote (broken down by supplier trade) should be accepted. Present when the action is a successful-quote action (`IsSuccessful`), the `PROJECT_QUOTES_BY_SUPPLIER_TRADE` system setting is enabled, and the quote request belongs to a project (`ProjectId` is set). Always required.

Note: The Type string emitted by the describe for this input is `"RejectLinks"` (the wrong constant was used in the provider). The `Invoke` method reads the accepted sections value from `input.SectionsAccepted` (an `int`). Callers should submit this under `SectionsAccepted` for the selection to be processed correctly.

Payload property: `SectionsAccepted`

```
{
  "SectionsAccepted": 1
}
```

Values: `0` = unspecified; `1` = accept required works section only; `2` = accept both required and additional works sections.

Metadata: `OptionSelectMeta<int>`

SuccessfulQuoteId

The ID of the quote to mark as the successful (accepted) quote for this quote request. Present when the action is a successful-quote action (`IsSuccessful`).

Note: The `Type` string emitted by the describe for this input is `"OrderCapexCodeId"` (a copy-paste error in the provider). The payload property the `Invoke` method reads is `SuccessfulQuoteId`; callers should submit the value under `SuccessfulQuoteId` for the selection to be saved correctly.

Payload property: `SuccessfulQuoteId`

```
{
  "SuccessfulQuoteId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: `OptionSelectMeta<Guid>` (described under key `"OrderCapexCodeId"`)

★ SupplierIds

The IDs of the suppliers to include in the quote request send-out. Present when the action is a supplier-selection action (`IsSupplierSelect`).

Payload property: `SupplierIds`

```
{
  "SupplierIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "6ba7b810-9dad-11d1-80b4-00c04fd430c8"
  ]
}
```

Metadata: `SupplierSelectMeta`

UpdateWaterWorkStatusId

The new remedial status to apply to the water work record attached to the quote request. Present when the quote request has an attached water work (`WaterWorkId` is set) and the action has a water work status configured that is not applied automatically (i.e. `StatAutoStatus` or `NonStatAutoStatus` is false for the applicable statutory/non-statutory path).

Payload property: `UpdateWaterWorkStatusId`

```
{
  "UpdateWaterWorkStatusId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: `OptionSelectMeta<Guid>`

UserIds

The IDs of the users to include in an email notification. Present when the action is configured to email specific other users (`IsEmailOthers`).

Payload property: `UserIds`

```
{
  "UserIds": [
    "3fa85f64-5717-4562-b3fc-2c963f66afa6",
    "6ba7b810-9dad-11d1-80b4-00c04fd430c8"
  ]
}
```

Metadata: None.

PROJECT CHANGE REQUEST ACTIONS

This section covers all action inputs applicable to Project Change Request entities.

ChangeRequestId

The unique identifier of the change request the action is being performed against. This is used during both validation (to load the change request and check its status) and invocation (to load, update, and audit the change request).

Payload property: `ChangeRequestId`

```
{
  "ChangeRequestId": "3fa85f64-5717-4562-b3fc-2c963f66afa6"
}
```

Metadata: None.

Message

A free-text comment or note to accompany the action. Present when the invoking action is not a "new request" action (i.e. it is acting on an existing change request), or when the action has an email body and subject configured.

- When the action is not a new-request action, the message is appended to an audit entry recorded against the change request: `"{ActionName}: {Message}"`.
- When the action has an email body and subject configured, the message is substituted into the email content via the `{action comments}` merge tag.

Payload property: `Message`

```
{
  "Message": "Approved following review of cost impact and programme implications."
}
```

Metadata: None.